

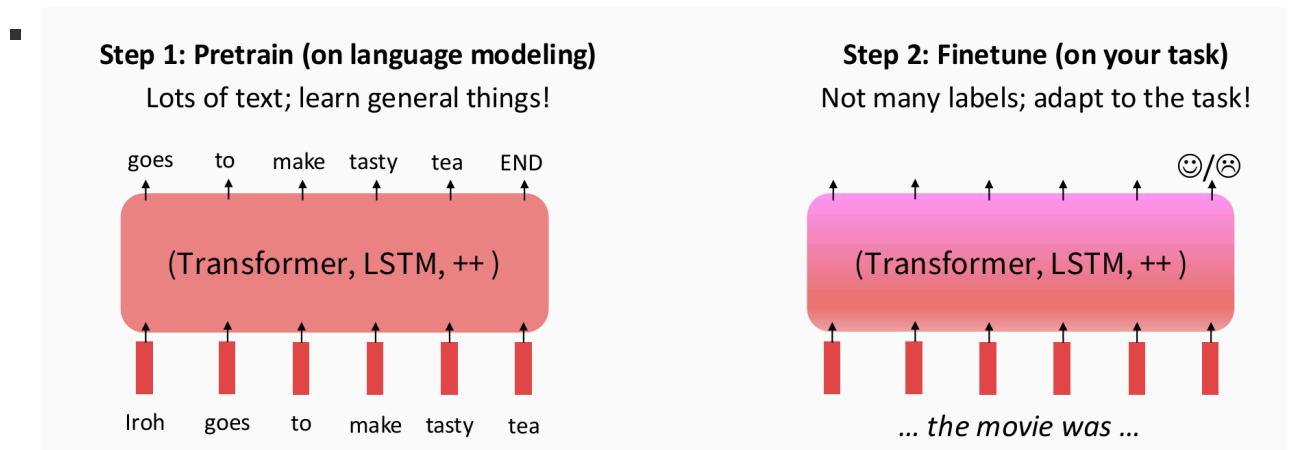
# Lecture 9

## 1. Word Structure and subword modelling

- From previous studies, we only have considered a fixed vocabulary, but for user to type in the prompts, their maybe misspellings or typos
- Similarly for a single word, there can be up to 300 conjugations, so if we consider all of them, that's too much to learn.
- **Fix: Subword Modelling**
  - learning the vocabulary for parts of words (subword token)
  - **Byte pair encoding**
    - we start with a vocabulary only containing characters and "end-of-word" symbols (separating words)
    - for unknown words, split them all into characters, and we find the most co-occurring adjacent characters.
    - then you replace those instances of occurring pairs with the subword.
    - and you keep doing this merging, until you reach a target vocabulary size

## 2. Pretraining

- **Previously** we pretrained the word embeddings (GloVe/Word2Vec), and the model is untrained (LSTM/RNN/Transformer), where their parameters are randomly initialized. Additionally no **Context** is learned. You only learn from the **downstream tasks (later given supervised data)**
- **Now** we train the **whole network** at pretraining.
  - it gives out strong representations of language
  - parameters are initialized stronger
  - strong probability distribution that we can sample from
  - you train everything **including the corpus** together, which means it is **self-supervised**
- **Pretraining through Language Modelling**
  - as we learned before, it gives  $p_{\theta}(w_t | w_{1:t-1})$  the probability of the next word given the previous words.
  - so you train through this over large amount of text, then you **save the network parameters**
  - **Paradigm**

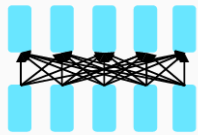


- so now when you do fine-tuning, your parameters are not randomized anymore
  - similarly when you do **SGD**, you don't start from a random parameter and start doing minimizing loss. Because it tends to stay within the basin/region near the initialization.
- Before, you just learn on downstream tasks, now you do it in fine tuning, referred to **SFT**. oooo~~

### 3. Model-Pretraining 3 ways

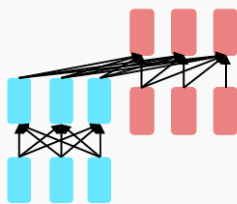
- Pretraining for three types of architectures

The neural architecture influences the type of pretraining, and natural use cases.



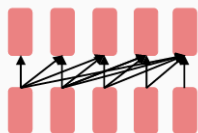
**Encoders**

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



**Encoder-Decoders**

- Good parts of decoders and encoders?
- What's the best way to pretrain them?



**Decoders**

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

- Encoders:**

- For **Encoders** you can train in a way such that you can Mask out fraction of the words, maintaining the bidirectional context.
  - $\tilde{x}$  is the masked version of  $x$
  - learning  $p_{\theta}(\tilde{x}|x)$
  - randomizing the masking part, then you can be learning.
  - BERT**

- they randomly chose 15% of the words to be predicted from the input, and for those chosen words, Mask 80% of it, and 10% replace the word with some random token, and the last 10% you keep it unchanged
- BERT was massively popular and hugely versatile; finetuning BERT led to new state-of-the-art results on a broad range of tasks.

- **QQP**: Quora Question Pairs (detect paraphrase questions)
- **QNLI**: natural language inference over question answering data
- **SST-2**: sentiment analysis
- **CoLA**: corpus of linguistic acceptability (detect whether sentences are grammatical.)
- **STS-B**: semantic textual similarity
- **MRPC**: microsoft paraphrase corpus
- **RTE**: a small natural language inference corpus

| System                | MNLI-(m/mm)<br>392k | QQP<br>363k | QNLI<br>108k | SST-2<br>67k | CoLA<br>8.5k | STS-B<br>5.7k | MRPC<br>3.5k | RTE<br>2.5k | Average     |
|-----------------------|---------------------|-------------|--------------|--------------|--------------|---------------|--------------|-------------|-------------|
| Pre-OpenAI SOTA       | 80.6/80.1           | 66.1        | 82.3         | 93.2         | 35.0         | 81.0          | 86.0         | 61.7        | 74.0        |
| BiLSTM+ELMo+Attn      | 76.4/76.1           | 64.8        | 79.8         | 90.4         | 36.0         | 73.3          | 84.9         | 56.8        | 71.0        |
| OpenAI GPT            | 82.1/81.4           | 70.3        | 87.4         | 91.3         | 45.4         | 80.0          | 82.3         | 56.0        | 75.1        |
| BERT <sub>BASE</sub>  | 84.6/83.4           | 71.2        | 90.5         | 93.5         | 52.1         | 85.8          | 88.9         | 66.4        | 79.6        |
| BERT <sub>LARGE</sub> | <b>86.7/85.9</b>    | <b>72.1</b> | <b>92.7</b>  | <b>94.9</b>  | <b>60.5</b>  | <b>86.5</b>   | <b>89.3</b>  | <b>70.1</b> | <b>82.1</b> |

- **Limitations**: would not be considering a **pretrained encoder** for generating token tasks. Good for filling in the blanks type of tasks.
- **Extentions** - spanBERT, where you mask out contiguous span of words.

## • Full fine-tuning VS Parameter-Efficient fine-tuning (PEFT) :

- where full fine tuning updates all the parameters when training, and the parameter efficient fine tuning , only choose a part of the parameters to update, keeping some good generality from the pretrained parameters.
- **2 types of Parameter-Efficient fine-tuning**
  - **Prefix Tuning, prompt tuning**: freezes all the pretrained parameters, and adds a prefix of parameters, then training that prefix. So in this way the old pretrained is kept, and while training only a small part is trained, not the whole gradients and optimizer states.
  - **LoRA**: freeze the weight matrix  $W$ , and difference in update  $AB$  which is very low **rank**, so overall would be  $W + AB$

## • Encoder-Decoder

- difference here would just be the generation process is generating a full sequence like language modelling, and the encoder part can read the text bidirectionally.
- **Best way to pre-train**: is to replace the original input with different length of span placeholders, and the decoder would be decoding the span out

- **Decoder**

- we can fine tune a decoder through a classifier based on the previous hidden states, then calculate gradient and backpropagate. Loop
- **pre-training** giving tasks of generating sequences with a vocabulary, just as when pretraining
- **In current age** most used are still decoders.

### 3. Optimization

#### Scaling Efficiency: how do we best use our compute

GPT-3 was **175B parameters** and trained on **300B** tokens of text.

Roughly, the cost of training a large transformer scales as **parameters\*tokens**

Did OpenAI strike the right parameter-token data to get the best model? No.

| Model                                              | Size (# Parameters) | Training Tokens |
|----------------------------------------------------|---------------------|-----------------|
| LaMDA ( <a href="#">Thoppilan et al., 2022</a> )   | 137 Billion         | 168 Billion     |
| GPT-3 ( <a href="#">Brown et al., 2020</a> )       | 175 Billion         | 300 Billion     |
| Jurassic ( <a href="#">Lieber et al., 2021</a> )   | 178 Billion         | 300 Billion     |
| <i>Gopher</i> ( <a href="#">Rae et al., 2021</a> ) | 280 Billion         | 300 Billion     |
| MT-NLG 530B ( <a href="#">Smith et al., 2022</a> ) | 530 Billion         | 270 Billion     |
| <i>Chinchilla</i>                                  | 70 Billion          | 1.4 Trillion    |

↖  
This 70B parameter model is better than the much larger other models!

- - not just about model size.
  - through CoT : referred as **In context Learning**